

Notes on Tree-based Methods

References:

- Chapter 8, An Introduction to Statistical Learning with Applications in R, Second Edition
- An intuitive explanation of gradient boosting by Richard Johansson
- A Gentle Introduction to Gradient Boosting by Cheng Li
- XGBoost: A Scalable Tree Boosting System by Tianqi Chen & Carlos Guestrin

Regression Trees

1. These methods involve stratifying or segmenting the predictor space into a number of simple regions. In order to make a prediction for a given observation, we typically use the mean or the mode response value for the training observations in the region to which it belongs. Decision trees can be applied to both regression and classification problems.
2. Roughly speaking, there are two steps in the process of building a regression tree:
 - a. We divide the predictor space – that is, the set of possible values for $X_1, X_2, \dots, X_p$ into J distinct and non-overlapping regions, $R_1, R_2, \dots, R_J$.
 - b. For every observation that falls into the region R_j , we make the same prediction, which is simply the mean of the response values for the training observations in R_j .
3. In theory, the regions could have any shape, but for simplicity and for ease of interpretation of the resulting predictive model, we choose to divide the predictor space into high-dimensional rectangles, or boxes. The goal is to find boxes $R_1, R_2, \dots, R_J$ that minimize the residual sum of squares (RSS), given by

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_i)^2,$$

where $\hat{y}_i$ is the mean response for the training observations within the j th box.

4. Unfortunately, it is computationally infeasible to consider every possible partition of the feature space into J boxes. For this reason, we take a top-down, greedy approach that is known as recursive binary splitting. In order to perform recursive binary splitting, we first select the predictor X_j and the cut point such that splitting the predictor space into the regions $\{X \mid X_j < s\}$ and $\{X \mid X_j \geq s\}$ leads to the greatest possible reduction in RSS.

5. Next, we repeat the process, looking for the best predictor and best cut point in order to split the data further so as to minimize the RSS within each of the resulting regions. However, this time, instead of splitting the entire predictor space, we split one of the two previously identified regions further, so as to minimize the RSS. The process continues until a stopping criterion is reached; for instance, we may continue until no region contains more than five observations.

Tree Pruning

1. The process described above may produce good predictions on the training set, but is likely to overfit the data, leading to poor test set performance. One alternative is to stop growing the tree whenever the best available split fails to reduce RSS by more than a specified threshold. However, this approach is too short-sighted because a split that produces only a small immediate reduction in RSS may create regions that can subsequently be split in a way that yields a much larger overall reduction in RSS.
2. Therefore, a better strategy is to grow a very large tree T_0 , and then prune it back in order to obtain a subtree. Pruning proceeds from the bottom of the tree upwards, successively removing leaf-level splits and creating a sequence of smaller trees. Intuitively, our goal is to select a subtree that leads to the lowest test error rate. Given a subtree, we can estimate its test error using cross-validation (many train-validation splits whose results are averaged) or the validation set approach (one train-validation split). However, estimating the cross-validation error for every possible subtree would be computationally infeasible, since the number of possible subtrees grows extremely rapidly as the size of the original tree increases. For example, a balanced binary decision tree with five levels of splits has more than 450,000 valid subtrees.
3. Cost-complexity pruning is analogous to the lasso in linear regression. Rather than selecting the tree that minimizes the training RSS alone, it minimizes the penalized objective

$$RSS(T) + \alpha|T|,$$

where $RSS(T)$ measures the tree's fit to the training data, $|T|$ is the number of terminal nodes (a measure of tree complexity), and α controls the trade-off between goodness of fit and model complexity. When $\alpha = 0$, there is no penalty for complexity and the fully grown tree T_0 is preferred; as α increases, larger trees become progressively less attractive. A useful way to think about the algorithm is to fix a value of α and repeatedly prune the weakest branch of the tree until the penalised objective ceases to decrease, arriving at the optimal

subtree for that value. As α increases from zero, the preferred subtree changes only at a finite number of values, generating a nested sequence

,

$$T_0 \supset T_1 \supset T_2 \supset \dots \supset T_K,$$

running from the fully grown tree down to the root alone. The trees in this sequence get smaller as α grows, so α is in effect choosing the size of the tree. This is what makes the search tractable: the number of possible subtrees is enormous — a balanced binary tree with five levels of splits has more than 450,000 — but we need only consider this much shorter sequence, since every tree omitted from it is one that no value of α would ever prefer.

It remains to choose α , and this cannot be done on the training data, where RSS falls monotonically with tree size and the largest tree always wins.

Cross-validation is used instead: the data are divided into folds, and for each fold a tree is grown on the remainder, pruned to produce its own sequence, and the tree preferred at each value of α is evaluated on the held-out fold. Averaging across folds gives an estimated test error for each α , and the value minimising it is selected. The final tree is the member of the full-data sequence corresponding to that α .

Trees Versus Linear Models

1. Whereas linear regression assumes a model of the form

$$f(X) = \beta_0 + \sum_{j=1}^p X_j \beta_j,$$

Regression trees assume a model of the form

$$f(X) = \sum_{m=1}^M c_m \cdot 1_{X \in R_m}$$

where $R_1, \dots, R_M$ represent a partition of feature space. If there is a highly non-linear and complex relationship between the features and the response, then the decision tree may outperform classical approaches.

Bagging, Random Forests, and Boosting

1. By aggregating many decision trees, using methods like bagging, random forests, and boosting, the predictive performance of trees can be substantially improved. An **ensemble method** is an approach that combines many simple “building block” models in order to obtain a single and potentially very powerful

model. These simple building block models are sometimes known as weak learners, since they may lead to mediocre predictions on their own.

2. The decision trees discussed above suffer from high variance, meaning that the resulting tree can differ significantly depending on how the full dataset is split into training, validation, and test sets. The intuition behind **bagging** (bootstrap aggregating) is that averaging multiple estimates reduces variance. The key idea is therefore to use bootstrapping to generate multiple training datasets, fit a separate regression tree to each bootstrap sample, and then average the resulting predictions. The trees are typically grown deep and left unpruned, allowing each tree to capture as much structure in its bootstrap sample as possible.
3. Unfortunately, when we bag a large number of trees, it is no longer possible to represent the resulting statistical learning procedure using a single tree. A way around this problem is to derive an overall measure of predictor importance. For each tree in the ensemble, the reductions in RSS attributable to splits on a given predictor are summed across all nodes where that predictor is used. These totals are then averaged across all trees. Predictors that consistently produce large **reductions in RSS** are deemed more important, since they contribute more to the predictive power of the bagged model.
4. **Random forests** improve upon bagging by way of a small but powerful modification. As in bagging, we build a large number of decision trees using bootstrapped training samples. However, when considering a split, the algorithm randomly selects only m predictors from the full set of p predictors and chooses the best split from this subset. A fresh random subset is selected at each node, with a common rule of thumb for regression trees being $m \approx p/3$. The intuition is that if one predictor, or a group of highly correlated predictors, is particularly strong, bagged trees will tend to look very similar because they repeatedly select those predictors near the top of the tree. Since averaging highly correlated predictions produces a smaller reduction in variance than averaging less correlated predictions, restricting the set of candidate predictors helps to decorrelate the trees. Choosing m therefore involves a trade-off: larger values tend to produce stronger individual trees but more highly correlated predictions, while smaller values increase tree diversity at the expense of weaker trees. In practice, m is often treated as a tuning parameter and selected using the out-of-bag (OOB) error estimate.
5. **Boosting** is another approach for improving predictions from decision trees, and like bagging it can be applied to regression or classification. For regression, the procedure is as follows:
 - Fit a small tree $\hat{f}_1$ — small in the sense that it has only a few terminal nodes, controlled by the depth parameter d (often $d = 1$, a stump). This is the model after the first step: $\hat{F}_1(x) = \hat{f}_1(x)$. Now compute the residuals $r_i =$

$y_i - \hat{F}_1(x_i)$, which record how much of each observation the model has failed to account for.

- Fit a second small tree $\hat{f}_2$ to those residuals rather than to y – its regions therefore predicting the mean residual of the observations falling in each – and add it, giving $\hat{F}_2(x) = \hat{f}_1(x) + \hat{f}_2(x)$. Recompute the residuals against this larger model, fit a third tree to them, and continue. Each tree targets the error left behind by everything added before it.
- After B steps the model is the accumulated sum of these corrections:

$$\hat{F}_B(x) = \sum_{b=1}^B \hat{f}_b(x).$$

Because every tree is fitted to what the previous ones left behind, the residuals fall steadily as trees are added and the model approaches y .

This is boosting, and it works as it stands. In practice, however, each tree is not adopted whole: only a fraction λ of it is added, so that the update becomes $\hat{F}_b(x) = \hat{F}_{b-1}(x) + \lambda \hat{f}_b(x)$ and the final model is $\hat{F}_B(x) = \sum_{b=1}^B \lambda \hat{f}_b(x)$.

The residuals are computed against this shrunk model at every stage, as before. Shrinkage slows the rate at which the model fits the data. Empirically this improves predictions on unseen data across a wide range of problems, at the cost of requiring more trees, with the trade-off between λ and B settled by cross-validation.

The algorithm has three tuning parameters: the number of trees B , the shrinkage λ (smaller λ requiring larger B), and the depth d . With $d = 1$ every tree is a stump and the boosted model is purely additive — a sum of one-variable step functions, which can be arbitrarily nonlinear but cannot express interactions such as "the effect of house size on price depends on location". Larger d allows up to d -way interactions within each tree, and hence across the ensemble.

Gradient Boosting

1. The algorithm just described minimises the squared error, which makes sense for a regression problem. But other losses may be preferred — the absolute error, which is less sensitive to outliers, or the negative log-likelihood of a probability model. Gradient boosting generalises the algorithm so that the ensemble can be optimised with respect to any of them.
2. To see how this generalisation is possible, it helps to recognise that the boosting procedure described above is already a form of gradient descent — and that squared error is already the loss it is descending. Gradient descent minimises a loss by repeated small adjustments to whatever quantity the loss depends on. Writing x for that quantity, each adjustment is:

$$x \leftarrow x - \lambda \nabla \text{Loss}(x),$$

where the arrow denotes replacement rather than equality: the value on the right becomes the new x , and the rule is applied again. Here $\nabla \text{Loss}(x)$ is the gradient of the loss, meaning its derivative with respect to x , which measures how sharply the loss responds to a change in x and in which direction it rises. Subtracting it therefore moves x in the direction that lowers the loss, by an amount governed by the step length λ (often written η in the gradient descent literature). Now observe that the negative gradient of half the squared error loss, evaluated at the fitted value $\hat{y}$, is the residual — that is, with $L(y_i, \hat{y}) = \frac{1}{2}(y_i - \hat{y})^2$, $-\partial L / \partial \hat{y} = y_i - \hat{y}$. So the boosting algorithm can be seen as a form of gradient descent that minimises the squared error loss, since each tree is fitted to the negative gradient of that loss. Gradient descent will already be familiar from linear models and neural networks; the difference here is that the model is improved by adding new sub-models rather than by changing weights.

This suggests how to generalise. For any other loss, compute its negative gradient at each observation and fit the tree to those values instead. Since they are no longer leftover errors in any literal sense, they are called pseudo-residuals.

3. The general algorithm follows the same recipe. For each training observation, compute the negative gradient of the loss at the current fitted value; fit a small regression tree to those values; add a fraction λ of it to the ensemble; repeat.

Written out:

- Let $\hat{F}_0$ be a constant model, for instance the mean of the response values.
- For $b = 1, \dots, B$:
 - Compute the pseudo-residual $r_i = -\partial L(y_i, \hat{F}_{b-1}(x_i)) / \partial \hat{F}_{b-1}(x_i)$ for each observation.
 - Fit a regression tree to the pseudo-residuals, which determines the regions $R_1, \dots, R_M$.
 - Every observation in a region is assigned the same value c_m , so choose it to make the total loss over the observations in that region as small as possible, giving $\hat{f}_b(x) = \sum_m c_m 1_{x \in R_m}$.
 - Add it to the ensemble as $\hat{F}_b(x) = \hat{F}_{b-1}(x) + \lambda \hat{f}_b(x)$.
- Return $\hat{F}_B(x)$.

The differences from the boosting algorithm of the previous section are confined to two lines: what the tree is fitted to, and how each region's value is set. The tree fitting, the shrinkage and the ensemble are unchanged, and the sub-models remain ordinary regression trees grown by recursive splitting on RSS, indifferent to which loss produced their target pseudo-residuals. Under half the squared error loss the algorithm collapses back to the one above: the pseudo-residuals

are the residuals, and the value minimising the loss over a region is the mean of the residuals falling there — since the constant closest to a set of numbers in the squared error sense is their average. That is what a regression tree assigns to a region anyway, so the earlier algorithm is recovered exactly.

4. Absolute error illustrates why one might want a different loss. Squared error grows quadratically with the size of the error, so a single badly aberrant observation can dominate the fitting, and the model will distort itself to accommodate it at the expense of the rest. The absolute error $L(y_i, \hat{y}) = |y_i - \hat{y}|$ grows only linearly and is correspondingly less sensitive to outliers.

The effect on the algorithm is confined, as expected, to the pseudo-residuals. Differentiating gives $-\partial L / \partial \hat{y} = \text{sign}(y_i - \hat{y})$, so the pseudo-residuals are now +1 or -1, recording only the direction of each error and not its size.

5. The same reasoning extends to likelihoods, which is where most alternative losses come from. A likelihood states how probable the observed data are under a probability model, so its negative logarithm is a measure of misfit, smaller when the model assigns the outcome higher probability. Maximising a likelihood and minimising a loss are therefore the same exercise, and any distributional assumption about the response supplies a loss ready for use.
6. Squared error is itself an instance. If the response is taken to be normally distributed about the model's prediction, independently across observations, then maximising the resulting likelihood is equivalent to minimising the residual sum of squares. Boosting with squared error has therefore been maximising a Gaussian likelihood all along. Nothing about least squares requires the data to be Gaussian; the point is that the objective can be read as a likelihood, and once it can, the likelihood becomes the component to exchange. A binary response would call for a binomial likelihood, a count for a Poisson, and in each case the algorithm is unchanged apart from the pseudo-residuals.
7. One feature of all these losses, including the likelihood-based ones, is worth noting, since it will not survive the final section. Each is a sum of one term per observation, so the pseudo-residual for an observation depends on that observation alone.
8. For most losses there is no formula for the value minimising the loss over a region, and it must be found numerically. XGBoost avoids this by minimising a quadratic approximation to the loss instead, which does have a closed form — the subject of the next section.

XGBoost

1. XGBoost — short for eXtreme Gradient Boosting — is an implementation of the gradient boosting algorithm described in the previous note, created by Tianqi Chen and Carlos Guestrin in 2014. The core is written in C++ for performance, with native bindings for Python, R, Java and Scala, and it is open source under the Apache-2.0 licence. Its reputation was made in competition: seventeen of the twenty-nine winning Kaggle solutions in 2015 used it, and by 2020 it accounted for the majority of winning solutions in tabular data competitions.
2. Several features distinguish XGBoost from a direct implementation of the algorithm above:
 - a. **Second-order optimisation, and custom objectives.** The value assigned to each region is found by minimising a quadratic approximation to the loss rather than the loss itself, which gives a closed form and removes the numerical search the previous section left hanging. Because the approximation is built from the first and second derivatives of the loss, a user wishing to supply a loss of their own must provide both — which is what makes the library usable for objectives none of its built-in choices covers.
 - b. **Regularisation.** Penalties are applied to the number of regions in a tree and to the size of the values they hold, so complexity is controlled within the fitting rather than only by limiting depth.
 - c. **Missing values.** A split sends an observation left or right according to whether its value for the splitting variable falls below the cut point, which is impossible if that value is absent. Each split therefore carries a default direction as well as a cut point, chosen during fitting by trying both and keeping whichever reduces the loss more. Observations with a missing value are sent that way. No imputation is required, and because such observations are routed together, the tree can treat the absence itself as informative.
 - d. **Efficiency.** A range of implementation choices — in how candidate splits are searched, how data is held in memory, and how work is spread across processors — make the algorithm fast on large datasets without changing what is fitted.

Applications to Horse Racing

1. Every loss considered so far is a sum of one term per observation, so the pseudo-residual for an observation depends on that observation alone. Racing breaks this. The natural unit is the race rather than the horse, and a single term of the loss spans every runner in the field.

2. The likelihood is the one derived in the earlier papers. Race i is the chooser and horse j the alternative; each horse carries a score z_{ij} , and win probabilities are given by the softmax

$$p_{ij} = \exp(z_{ij}) / \sum_m \exp(z_{im}),$$

whose denominator runs over the runners in race i . Letting $j_1, \dots, j_k$ denote the top k finishers in order, and writing $\mathcal{C}_{is} = \{j_s, j_{s+1}, \dots, j_j\}$ for the horses not yet ranked at position s , the Plackett–Luce log-likelihood over a set of races is

$$\sum_{i=1}^n \sum_{s=1}^k \left(z_{ij_s} - \log \sum_{j \in \mathcal{C}_{is}} \exp(z_{ij}) \right).$$

This is the log of the probability the model assigns to the order that actually occurred: at each position s , the first term is the score of the horse that in fact finished there, and the second sums over the horses still unranked. The observed finishing order therefore enters the loss through which horses appear in which position. Taking $k = 1$ recovers the conditional logit, in which only the winner is modelled. The loss to be minimised is the negative of this.

3. The finishing order is what is observed. The scores z_{ij} are not: they are what the ensemble produces for each horse, standing in the position that fitted values occupy in a regression, but unlike a fitted value a score is not an estimate of anything observable — it is a latent measure of ability, and its only role is to feed the softmax. Each tree is fitted with the usual explanatory variables as predictors and the pseudo-residuals as the response, which determines how the horses are grouped into regions; the value assigned to each region is then whatever amount, added to the scores of the horses falling in it, makes the loss over those horses as small as possible. Ability is thus inferred from results: horses given too little probability by the current scores have their scores raised, and the ordering of past races determines, indirectly, what score a horse with a given set of characteristics receives.

4. A further advantage is worth noting. Consider a branch that first splits on the *going* and then, further down, on the horse's win rate over the distance. Every horse reaching the second split has already satisfied the first, so the value at the end of that branch applies only to horses with a particular distance record on that going. The effect of the distance record is therefore allowed to differ by *going*, which is what an interaction means. A tree of depth d can express interactions among d variables in this way, and finds them from the data rather than needing to be told which to look for — an advantage where the effect of a horse's characteristics on its chance is unlikely to be additive.

5. In summary, the covariate space is partitioned into regions on which the score is constant. Because each split is a threshold on a single variable, those regions are rectangles with sides parallel to the axes: a horse in the same region as another is one that matches it on every threshold the trees have found, whatever their other differences. Within a race the scores are converted to win probabilities by the softmax, and those probabilities give the likelihood of the observed finishing order. The partition and the scores are built up over many rounds, each one adjusting the scores so as to raise that likelihood. Two horses in the same race generally fall in different regions, and it is that difference that the softmax turns into an ordering of the field.